Examen parcial 2 (simulación 1) — Respuestas

Nivel intermedio · 7 secciones × 3 preguntas · 3 opciones por pregunta. Cada alumno vio las opciones en otro orden, por eso aquí no van letras: la correcta está marcada con ✅. En los códigos, lo resaltado en amarillo es lo que cambia respecto a la versión correcta.

Sección	Cómo se califica
1–3, 6 (opción múltiple)	1 punto si elige la correcta
4–5 (bugs)	1 punto si el código corregido pasa todas las pruebas (visible y ocultas)
7 (problema abierto)	Fracción de pruebas que pasa (visibles + ocultas)
Calificación	puntos / 21 × 10

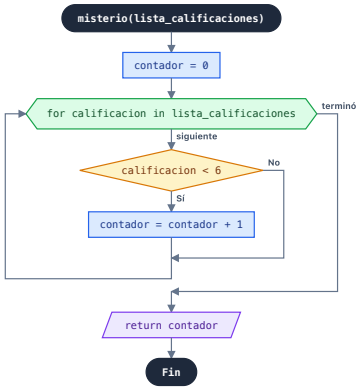
1. Ponle nombre a la función

Lee el código y sus dos diagramas. Elige el nombre que dice qué hace la función y que además sigue la convención del curso: verbo en infinitivo + objeto (+ contexto); es_… sólo si regresa True / False.

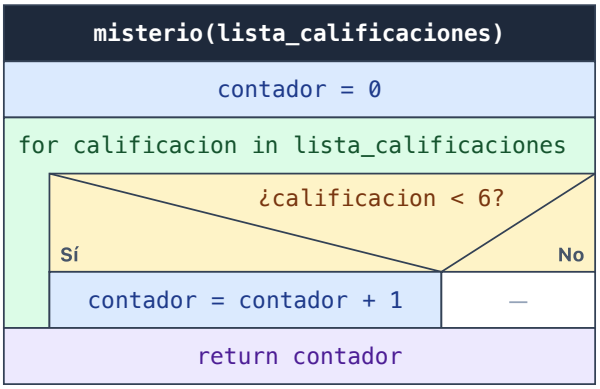
Pregunta 1.1

```
def misterio(lista_calificaciones):
    contador = 0
    for calificacion in lista_calificaciones:
        if calificacion < 6:
            contador = contador + 1
    return contador
```

Diagrama de flujo



Nassi-Shneiderman



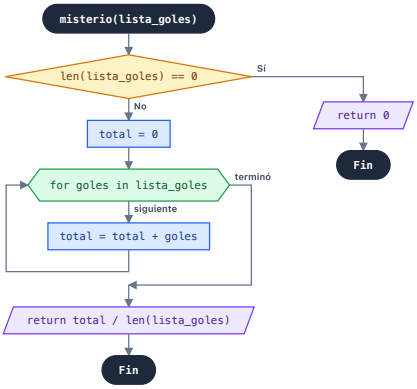
✅ `contar_reprobados` ❌ `contar_aprobados` ❌ `numero_reprobados`

> 💡 Cuenta las calificaciones menores que 6: los reprobados. numero_reprobados no tiene verbo.

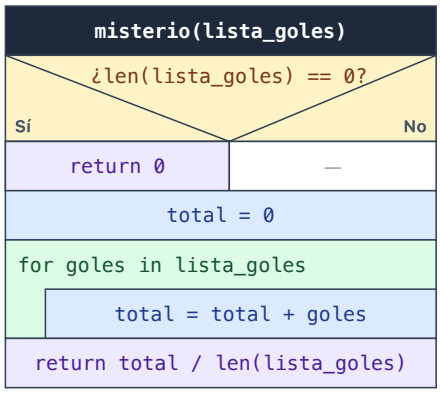
Pregunta 1.2

```
def misterio(lista_goles):
    if len(lista_goles) == 0:
        return 0
    total = 0
    for goles in lista_goles:
        total = total + goles
    return total / len(lista_goles)
```

Diagrama de flujo



Nassi-Shneiderman



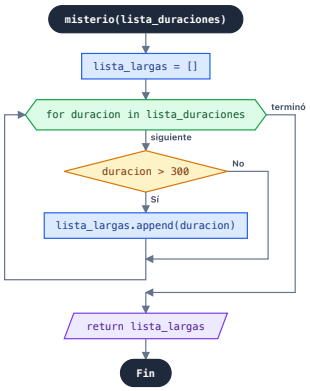
✅ **`calcular_promedio_goles`** ❌ `calcular_total_goles` ❌ `calcular_goles_promedio`

> 💡 Al final divide entre la cantidad de partidos: es un promedio, no un total. El objeto (promedio) va pegado al verbo y luego el contexto (goles).

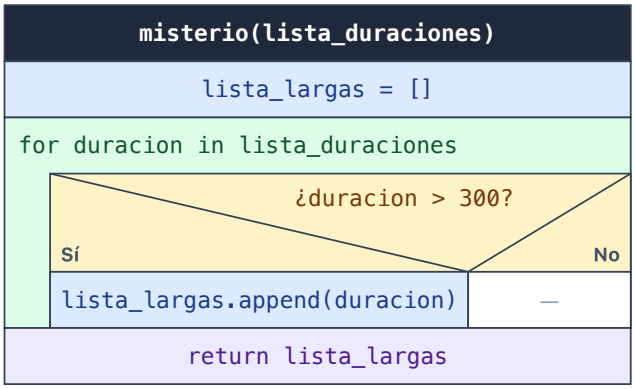
Pregunta 1.3

```
def misterio(lista_duraciones):
    lista_largas = []
    for duracion in lista_duraciones:
        if duracion > 300:
            lista_largas.append(duracion)
    return lista_largas
```

Diagrama de flujo



Nassi-Shneiderman



✅ **`filtrar_canciones_largas`** ❌ `contar_canciones_largas` ❌ `es_cancion_larga`

> 💡 Regresa una lista NUEVA con las duraciones mayores que 300: filtra, no cuenta. es_... se usa sólo cuando la función regresa True / False.

2. Encuentra los diagramas del código

Las tres opciones tienen las mismas instrucciones que el código, pero en otro orden o dentro de otro bloque. Elige el par de diagramas (flujo y Nassi-Shneiderman) que representa exactamente al código.

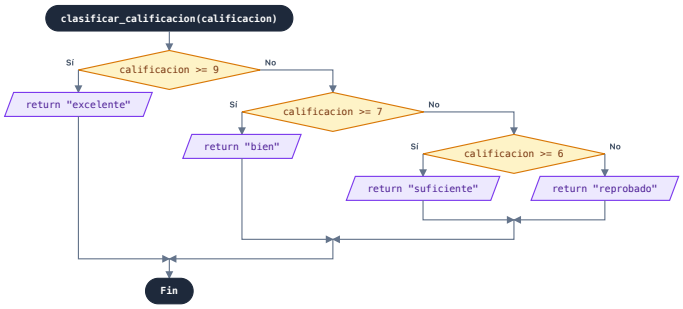
Pregunta 2.1

Código

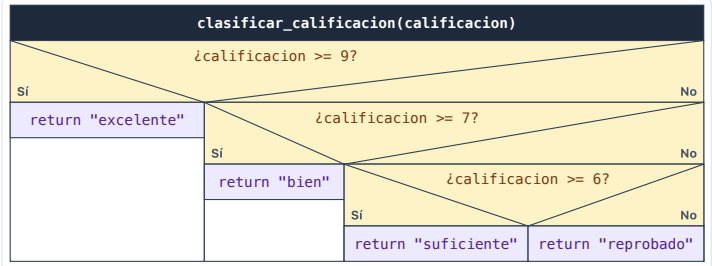
```
def clasificar_calificacion(calificacion):
    if calificacion >= 9:
        return "excelente"
    elif calificacion >= 7:
        return "bien"
    elif calificacion >= 6:
        return "suficiente"
    else:
        return "reprobado"
```

✅ **Diagramas correctos**

Diagrama de flujo



Nassi-Shneiderman



❌ **Las otras opciones dibujaban este código (instrucciones en otro orden):**

```
def clasificar_calificacion(calificacion):
    if calificacion >= 6:
        return "suficiente"
    elif calificacion >= 9:
        return "excelente"
    elif calificacion >= 7:
        return "bien"
    else:
        return "reprobado"
```

```
def clasificar_calificacion(calificacion):
    if calificacion >= 7:
        return "bien"
    elif calificacion >= 9:
        return "excelente"
    elif calificacion >= 6:
        return "suficiente"
    else:
        return "reprobado"
```

💡 Las decisiones van de la más exigente (>= 9) a la menos exigente (>= 6), en ese orden.

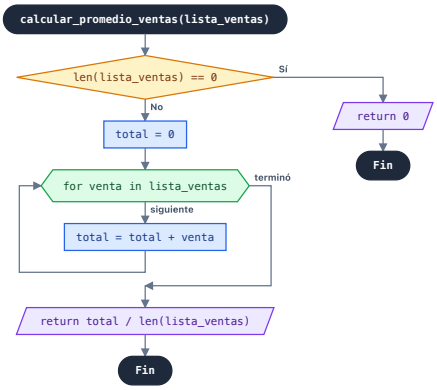
Pregunta 2.2

Código

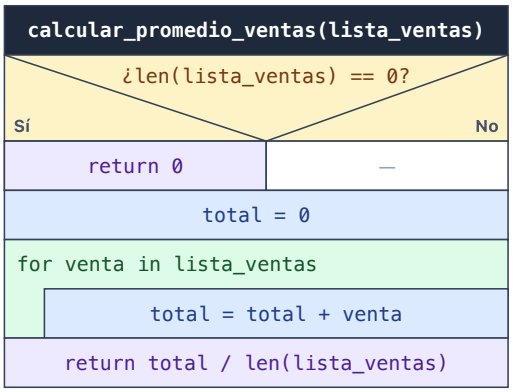
```
def calcular_promedio_ventas(lista_ventas):
    if len(lista_ventas) == 0:
        return 0
    total = 0
    for venta in lista_ventas:
        total = total + venta
    return total / len(lista_ventas)
```

✅ **Diagramas correctos**

Diagrama de flujo



Nassi-Shneiderman



❌ **Las otras opciones dibujaban este código (instrucciones en otro orden):**

```
def calcular_promedio_ventas(lista_ventas):
    total = 0
    for venta in lista_ventas:
        if len(lista_ventas) == 0:
            return 0
        total = total + venta
    return total / len(lista_ventas)
```

```
def calcular_promedio_ventas(lista_ventas):
    if len(lista_ventas) == 0:
        return 0
    total = 0
    for venta in lista_ventas:
        total = total + venta
        return total / len(lista_ventas)
```

💡 El edge case (lista vacía) se revisa al inicio, fuera del ciclo, y la división va después del ciclo.

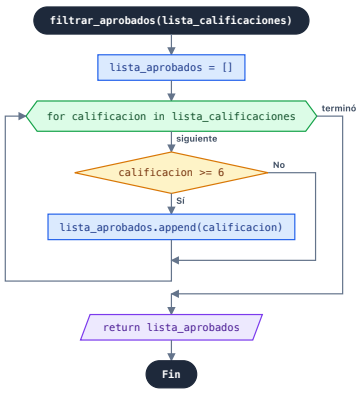
Pregunta 2.3

Código

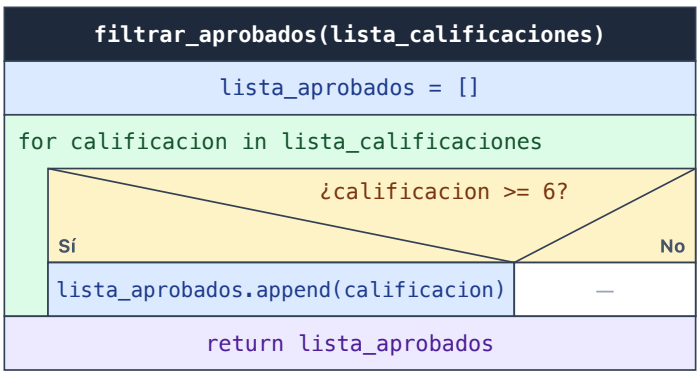
```
def filtrar_aprobados(lista_calificaciones):
    lista_aprobados = []
    for calificacion in lista_calificaciones:
        if calificacion >= 6:
            lista_aprobados.append(calificacion)
    return lista_aprobados
```

✅ **Diagramas correctos**

Diagrama de flujo



Nassi-Shneiderman



❌ **Las otras opciones dibujaban este código (instrucciones en otro orden):**

```
def filtrar_aprobados(lista_calificaciones):
    for calificacion in lista_calificaciones:
        lista_aprobados = []
        if calificacion >= 6:
            lista_aprobados.append(calificacion)
    return lista_aprobados
```

```
def filtrar_aprobados(lista_calificaciones):
    lista_aprobados = []
    for calificacion in lista_calificaciones:
        if calificacion >= 6:
            lista_aprobados.append(calificacion)
        return lista_aprobados
```

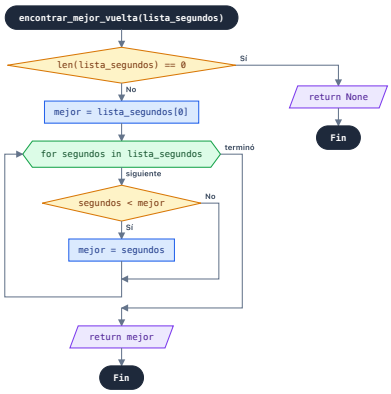
💡 La lista nueva se crea una vez, antes del for, y el append va dentro del if.

3. Encuentra el código de los diagramas

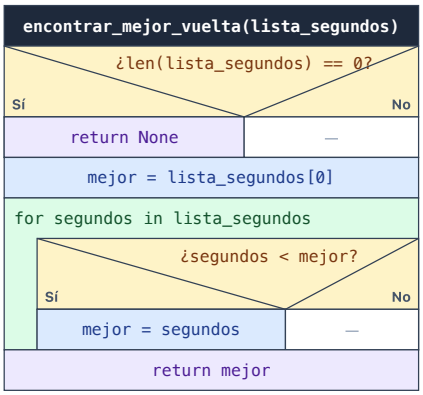
Los tres códigos tienen las mismas instrucciones, pero en otro orden o con otra sangría. Elige el que representa exactamente a los diagramas.

Pregunta 3.1

Diagrama de flujo



Nassi-Shneiderman



✅ Correcta

```
def encontrar_mejor_vuelta(lista_segundos):
    if len(lista_segundos) == 0:
        return None
    mejor = lista_segundos[0]
    for segundos in lista_segundos:
        if segundos < mejor:
            mejor = segundos
    return mejor
```

❌ Incorrecta

```
def encontrar_mejor_vuelta(lista_segundos):
    mejor = lista_segundos[0]
    if len(lista_segundos) == 0:
        return None
    for segundos in lista_segundos:
        if segundos < mejor:
            mejor = segundos
    return mejor
```

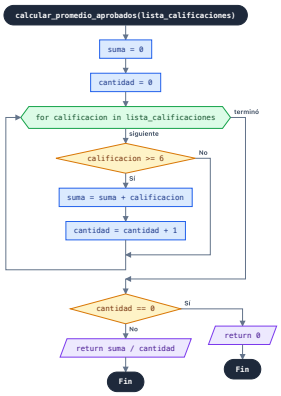
❌ Incorrecta

```
def encontrar_mejor_vuelta(lista_segundos):
    if len(lista_segundos) == 0:
        return None
    mejor = lista_segundos[0]
    for segundos in lista_segundos:
        if segundos < mejor:
            mejor = segundos
            return mejor
```

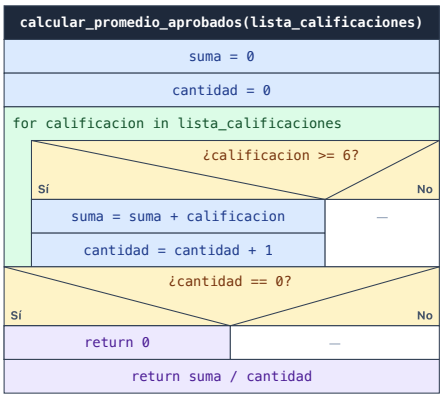
💡 Primero el edge case, luego el valor inicial (lista_segundos[0]) y el return al final, fuera del ciclo.

Pregunta 3.2

Diagrama de flujo



Nassi-Shneiderman



✅ Correcta

```
def calcular_promedio_aprobados(lista_calificaciones):
    suma = 0
    cantidad = 0
    for calificacion in lista_calificaciones:
        if calificacion >= 6:
            suma = suma + calificacion
            cantidad = cantidad + 1
    if cantidad == 0:
        return 0
    return suma / cantidad
```

❌ Incorrecta

```
def calcular_promedio_aprobados(lista_calificaciones):
    suma = 0
    cantidad = 0
    for calificacion in lista_calificaciones:
        if calificacion >= 6:
            suma = suma + calificacion
        cantidad = cantidad + 1
    if cantidad == 0:
        return 0
    return suma / cantidad
```

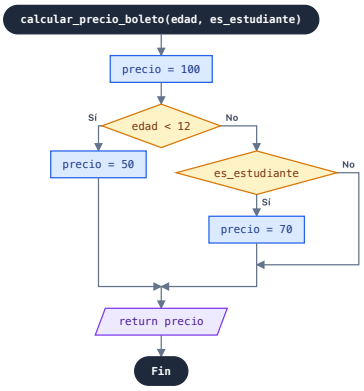
❌ Incorrecta

```
def calcular_promedio_aprobados(lista_calificaciones):
    suma = 0
    cantidad = 0
    for calificacion in lista_calificaciones:
        if calificacion >= 6:
            suma = suma + calificacion
            cantidad = cantidad + 1
        if cantidad == 0:
            return 0
    return suma / cantidad
```

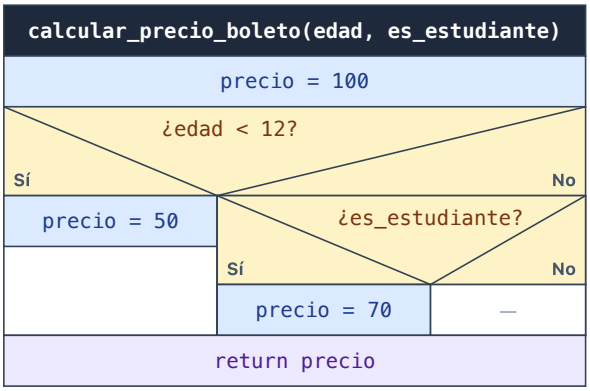
💡 Los dos acumuladores aumentan dentro del if (sólo los aprobados) y la validación de cantidad == 0 va después del ciclo.

Pregunta 3.3

Diagrama de flujo



Nassi-Shneiderman



Correcta

```
def calcular_precio_boleto(edad, es_estudiante):
    precio = 100
    if edad < 12:
        precio = 50
    elif es_estudiante:
        precio = 70
    return precio
```

Incorrecta

```
def calcular_precio_boleto(edad, es_estudiante):
    if edad < 12:
        precio = 50
    elif es_estudiante:
        precio = 70
    precio = 100
    return precio
```

Incorrecta

```
def calcular_precio_boleto(edad, es_estudiante):
    precio = 100
    if es_estudiante:
        precio = 70
    elif edad < 12:
        precio = 50
    return precio
```

El precio normal se asigna antes del if, y la primera pregunta es la de la edad (edad < 12).

4. Encuentra el bug: una instrucción fuera de lugar

Cada código tiene UNA instrucción en el lugar equivocado (antes o después de donde va, o dentro / fuera de un bloque). No hay errores de sintaxis ni de valores: sólo mueve la instrucción a su lugar. En el código no se escribe: haz clic en una línea y súbela, bájala o cámbiale la sangría con los botones; en los diagramas, arrastra los bloques. Aquí no se puede ejecutar: haz la prueba de escritorio en tu hoja.

Pregunta 4.1 · Fitbit: promedio de calorías

En Fitbit, la función recibe las calorías quemadas cada día y regresa el promedio. Si la lista está vacía, debe regresar `0`.

Con la lista vacía `[]` el programa se detiene con `ZeroDivisionError: division by zero` en lugar de regresar `0`.

Código con el bug

```
def calcular_promedio_calorias(lista_calorias):
    total = 0
    for calorias in lista_calorias:
        total = total + calorias
    promedio = total / len(lista_calorias)
    if len(lista_calorias) == 0:
        return 0
    return promedio
```

Corregido

```
def calcular_promedio_calorias(lista_calorias):
    if len(lista_calorias) == 0:
        return 0
    total = 0
    for calorias in lista_calorias:
        total = total + calorias
    promedio = total / len(lista_calorias)
    return promedio
```

Pruebas (visible y ocultas)

```
calcular_promedio_calorias([]) => 0  # ✗ da otro resultado: aquí está el bug
calcular_promedio_calorias([300, 500]) => 400.0
calcular_promedio_calorias([250]) => 250.0
calcular_promedio_calorias([100, 200, 600]) => 300.0
```

> 💡 El edge case se valida ANTES de usar los datos: el if va al inicio de la función.

Pregunta 4.2 · Aeroméxico: vuelos retrasados

En Aeroméxico cada vuelo es un diccionario como `{"codigo": "AM100", "retraso": 30}` (el retraso en minutos). La función debe regresar una lista con los códigos de los vuelos con más de 15 minutos de retraso.

Con `[{"codigo": "AM100", "retraso": 30}, {"codigo": "AM200", "retraso": 0}]` se esperaba `["AM100"]`, pero regresa `["AM100", "AM100"]`.

Código con el bug

```
def filtrar_vuelos_retrasados(lista_vuelos):
    lista_codigos = []
    for vuelo in lista_vuelos:
        if vuelo["retraso"] > 15:
            codigo = vuelo["codigo"]
        lista_codigos.append(codigo)
    return lista_codigos
```

Corregido

```
def filtrar_vuelos_retrasados(lista_vuelos):
    lista_codigos = []
    for vuelo in lista_vuelos:
        if vuelo["retraso"] > 15:
            codigo = vuelo["codigo"]
            lista_codigos.append(codigo)
    return lista_codigos
```

Pruebas (visible y ocultas)

```
filtrar_vuelos_retrasados([{"codigo": "AM100", "retraso": 30}, {"codigo": "AM200", "retraso": 0}]) => ["AM100"]  # ✗ da otro resultado: aquí está el bug
filtrar_vuelos_retrasados([]) => []
filtrar_vuelos_retrasados([{"codigo": "AM300", "retraso": 5}]) => []
filtrar_vuelos_retrasados([{"codigo": "AM1", "retraso": 16}, {"codigo": "AM2", "retraso": 40}]) => ["AM1", "AM2"]
```

> 💡 El append estaba fuera del if: agregaba un código en cada vuelta, aunque el vuelo no estuviera retrasado.

Pregunta 4.3 · Uber: cambio de turno

En Uber, al cambiar de turno, los dos conductores asignados a una ruta se intercambian: el que iba primero pasa a segundo y viceversa. La función recibe los dos nombres y regresa una lista con los nombres ya intercambiados.

Con `"Ana"` y `"Beto"` se esperaba `["Beto", "Ana"]`, pero regresa `["Ana", "Ana"]`.

Código con el bug

```
def intercambiar_conductores(primero, segundo):
    auxiliar = primero
    segundo = auxiliar
    primero = segundo
    return [primero, segundo]
```

Corregido

```
def intercambiar_conductores(primero, segundo):
    auxiliar = primero
    primero = segundo
    segundo = auxiliar
    return [primero, segundo]
```

Pruebas (visible y ocultas)

```
intercambiar_conductores("Ana", "Beto") => ["Beto", "Ana"]  # ✗ da otro resultado: aquí está el bug
intercambiar_conductores("Caro", "Dani") => ["Dani", "Caro"]
intercambiar_conductores("Eva", "Eva") => ["Eva", "Eva"]
```

> 💡 Con la auxiliar, el orden es: guardar primero en la auxiliar, luego primero recibe a segundo y al final segundo recibe la auxiliar.

5. Encuentra el bug: condición, índice, valor inicial...

El orden de las instrucciones es correcto, pero hay UN error: una comparación (<, >, <=, >=), un True / False al revés, un elif donde va else, un índice o una llave equivocados, un acumulador mal inicializado o que acumula mal... Corrígelo en el código o en los diagramas. Aquí no se puede ejecutar: haz la prueba de escritorio en tu hoja.

Pregunta 5.1 · Amazon: el precio más bajo

En Amazon, la función recibe los precios de un producto en varias tiendas y regresa el más bajo. La lista nunca está vacía.

Con `[250, 199, 320]` se esperaba `199`, pero regresa `0`.

Código con el bug

```
def encontrar_minimo_precios(lista_precios):
    minimo = 0
    for precio in lista_precios:
        if precio < minimo:
            minimo = precio
    return minimo
```

Corregido

```
def encontrar_minimo_precios(lista_precios):
    minimo = lista_precios[0]
    for precio in lista_precios:
        if precio < minimo:
            minimo = precio
    return minimo
```

Pruebas (visible y ocultas)

```
encontrar_minimo_precios([250, 199, 320]) => 199  # ✗ da otro resultado: aquí está el bug
encontrar_minimo_precios([5]) => 5
encontrar_minimo_precios([10, 20]) => 10
encontrar_minimo_precios([30, 20, 10]) => 10
```

> 💡 El mínimo empezaba en 0 y ningún precio es menor que 0: debe empezar con el primer elemento.

Pregunta 5.2 · Metro: tarifa por edad

En el Metro, los menores de 5 años no pagan (`0`), las personas de 60 años o más pagan `2` y todos los demás pagan `5`.

Con una edad de `5` se esperaba `5`, pero regresa `None`.

Código con el bug

```
def calcular_tarifa_metro(edad):
    if edad < 5:
        return 0
    elif edad >= 60:
        return 2
    elif edad > 5:
        return 5
```

Corregido

```
def calcular_tarifa_metro(edad):
    if edad < 5:
        return 0
    elif edad >= 60:
        return 2
    else:
        return 5
```

Pruebas (visible y ocultas)

```
calcular_tarifa_metro(5) => 5  # ✗ da otro resultado: aquí está el bug
calcular_tarifa_metro(3) => 0
calcular_tarifa_metro(60) => 2
calcular_tarifa_metro(30) => 5
```

> 💡 "Todos los demás" es un else. Con elif edad > 5 el 5 no entraba en ningún caso.

Pregunta 5.3 · Spotify: la última canción

En Spotify, la función regresa la última canción de una playlist; si está vacía, regresa `None`.

Con `["Tusa", "Bichota"]` el programa se detiene con `IndexError: list index out of range` en lugar de regresar `"Bichota"`.

Código con el bug

```
def obtener_ultima_cancion(lista_canciones):
    if len(lista_canciones) == 0:
        return None
    return lista_canciones[len(lista_canciones)]
```

Corregido

```
def obtener_ultima_cancion(lista_canciones):
    if len(lista_canciones) == 0:
        return None
    return lista_canciones[len(lista_canciones) - 1]
```

Pruebas (visible y ocultas)

```
obtener_ultima_cancion(["Tusa", "Bichota"]) => "Bichota"  # ✗ da otro resultado: aquí está el bug
obtener_ultima_cancion([]) => None
obtener_ultima_cancion(["Provenza"]) => "Provenza"
obtener_ultima_cancion(["a", "b", "c"]) => "c"
```

> 💡 Las posiciones empiezan en 0: la última es len(lista) - 1.

6. Problema con opción múltiple

Lee el problema y elige la función que lo resuelve en TODOS los casos. No se puede ejecutar: haz la prueba de escritorio en tu hoja con los ejemplos y con los casos especiales.

Pregunta 6.1 · Promedio del grupo

Regresa el promedio de las calificaciones de un grupo. Si la lista está vacía, regresa `0`.

Entrada: `lista_calificaciones`: lista de números (puede estar vacía).
Salida: Un número: el promedio, o `0` si no hay calificaciones.

Ejemplos

```
calcular_promedio_grupo([8, 9, 10]) => 9.0
calcular_promedio_grupo([7, 8]) => 7.5
```

Casos especiales (ocultos)

```
calcular_promedio_grupo([]) => 0
calcular_promedio_grupo([6]) => 6.0
```

✅ **Correcta**

```
def calcular_promedio_grupo(lista_calificaciones):
    if len(lista_calificaciones) == 0:
        return 0
    total = 0
    for calificacion in lista_calificaciones:
        total = total + calificacion
    return total / len(lista_calificaciones)
```

❌ **Incorrecta**

```
def calcular_promedio_grupo(lista_calificaciones):
    total = 0
    for calificacion in lista_calificaciones:
        total = total + calificacion
    promedio = total / len(lista_calificaciones)
    if len(lista_calificaciones) == 0:
        return 0
    return promedio
```

❌ **Incorrecta**

```
def calcular_promedio_grupo(lista_calificaciones):
    if len(lista_calificaciones) == 0:
        return 0
    total = 0
    for calificacion in lista_calificaciones:
        total = total + calificacion
    return total // len(lista_calificaciones)
```

💡 La segunda divide entre 0 con la lista vacía antes de validarla; la tercera usa // y con [7, 8] da 7 en lugar de 7.5.

Pregunta 6.2 · Productos agotados

Cada producto del inventario es un diccionario como `{"nombre": "Gansito", "existencias": 0}`. Regresa una lista con los **nombres** de los productos agotados (0 existencias), en el mismo orden.

Entrada: `lista_productos`: lista de diccionarios con las llaves `"nombre"` y `"existencias"`.
Salida: Una lista de textos.

Ejemplos

```
filtrar_productos_agotados([{"nombre": "Gansito", "existencias": 0}, {"nombre": "Takis", "existencias": 4}]) => ["Gansito"]
```

Casos especiales (ocultos)

```
filtrar_productos_agotados([]) => []
filtrar_productos_agotados([{"nombre": "A", "existencias": 0}, {"nombre": "B", "existencias": 0}]) => ["A", "B"]
```

✅ **Correcta**

```
def filtrar_productos_agotados(lista_productos):
    lista_agotados = []
    for producto in lista_productos:
        if producto["existencias"] == 0:
            lista_agotados.append(producto["nombre"])
    return lista_agotados
```

❌ **Incorrecta**

```
def filtrar_productos_agotados(lista_productos):
    lista_agotados = []
    for producto in lista_productos:
        if producto["existencias"] == 0:
            lista_agotados.append(producto)
    return lista_agotados
```

❌ **Incorrecta**

```
def filtrar_productos_agotados(lista_productos):
    for producto in lista_productos:
        lista_agotados = []
        if producto["existencias"] == 0:
            lista_agotados.append(producto["nombre"])
    return lista_agotados
```

💡 La segunda agrega el diccionario completo (no el nombre) y la tercera crea la lista en cada vuelta (y truena con la lista vacía).

Pregunta 6.3 · Índice de masa corporal

Clasifica un IMC: menos de 18.5 es `"bajo"`; menos de 25, `"normal"`; menos de 30, `"sobrepeso"`; 30 o más, `"obesidad"`.

Entrada: `imc`: número mayor que 0.
Salida: Un texto con la clasificación.

Ejemplos

```
clasificar_imc(17) => "bajo"
clasificar_imc(22) => "normal"
clasificar_imc(35) => "obesidad"
```

Casos especiales (ocultos)

```
clasificar_imc(25) => "sobrepeso"
clasificar_imc(18.5) => "normal"
clasificar_imc(29.9) => "sobrepeso"
```

✅ **Correcta**

```
def clasificar_imc(imc):
    if imc < 18.5:
        return "bajo"
    elif imc < 25:
        return "normal"
    elif imc < 30:
        return "sobrepeso"
    else:
        return "obesidad"
```

❌ **Incorrecta**

```
def clasificar_imc(imc):
    if imc < 18.5:
        return "bajo"
    elif imc <= 25:
        return "normal"
    elif imc <= 30:
        return "sobrepeso"
    else:
        return "obesidad"
```

❌ **Incorrecta**

```
def clasificar_imc(imc):
    if imc < 30:
        return "sobrepeso"
    elif imc < 25:
        return "normal"
    elif imc < 18.5:
        return "bajo"
    else:
        return "obesidad"
```

💡 Con <= el 25 queda como normal. En la tercera, todo lo menor que 30 entra en el primer if, así que 17 y 22 salen como sobrepeso. Prueba los valores frontera (18.5, 25, 30).

7. Problema: escribe tu solución

Escribe la función. ▶ Probar corre sólo los ejemplos; al enviar el examen también se corren pruebas ocultas que no ves (casos especiales), así que piensa en ellos.

Pregunta 7.1 · Promedio de pasos

Tu app de salud guarda los pasos de cada día. Escribe una función que regrese el promedio de pasos. Si no hay días registrados, regresa `0`.

Entrada: `lista_pasos`: lista de enteros (puede estar vacía).
Salida: El promedio (con decimales), o `0` si la lista está vacía.

✅ **Una solución**

```
def calcular_promedio_pasos(lista_pasos):
    if len(lista_pasos) == 0:
        return 0
    total = 0
    for pasos in lista_pasos:
        total = total + pasos
    return total / len(lista_pasos)
```

Pruebas visibles (▶ Probar)

```
calcular_promedio_pasos([8000, 10000]) => 9000.0
calcular_promedio_pasos([1000, 2000, 6000]) => 3000.0
```

Pruebas ocultas (al enviar)

```
calcular_promedio_pasos([]) => 0  # sin días registrados
calcular_promedio_pasos([5000]) => 5000.0
calcular_promedio_pasos([1, 2]) => 1.5  # con decimales
```

Pregunta 7.2 · Películas cortas

En Netflix cada película es un diccionario como `{"titulo": "Coco", "minutos": 105}`. Escribe una función que regrese una lista con los **títulos** de las películas de 100 minutos o menos, en el mismo orden.

Entrada: `lista_peliculas`: lista de diccionarios con las llaves `"titulo"` y `"minutos"`.
Salida: Una lista de textos (vacía si ninguna cumple).

✅ **Una solución**

```
def filtrar_peliculas_cortas(lista_peliculas):
    lista_titulos = []
    for pelicula in lista_peliculas:
        if pelicula["minutos"] <= 100:
            lista_titulos.append(pelicula["titulo"])
    return lista_titulos
```

Pruebas visibles (▶ Probar)

```
filtrar_peliculas_cortas([{"titulo": "Coco", "minutos": 105}, {"titulo": "Up", "minutos": 96}]) => ["Up"]
```

Pruebas ocultas (al enviar)

```
filtrar_peliculas_cortas([]) => []  # sin películas
filtrar_peliculas_cortas([{"titulo": "A", "minutos": 100}]) => ["A"]  # justo 100
filtrar_peliculas_cortas([{"titulo": "A", "minutos": 90}, {"titulo": "B", "minutos": 80}]) => ["A", "B"]
```

Pregunta 7.3 · Radar de velocidad

Un radar clasifica la velocidad de un auto: hasta 60 km/h es `"normal"`, hasta 100 km/h es `"rápido"` y más de 100 km/h es `"exceso"`.

Entrada: `velocidad`: número de 0 en adelante.
Salida: Un texto: `"normal"`, `"rápido"` o `"exceso"`.

✅ **Una solución**

```
def clasificar_velocidad(velocidad):
    if velocidad <= 60:
        return "normal"
    elif velocidad <= 100:
        return "rápido"
    else:
        return "exceso"
```

Pruebas visibles (▶ Probar)

```
clasificar_velocidad(40) => "normal"
clasificar_velocidad(80) => "rápido"
clasificar_velocidad(130) => "exceso"
```

Pruebas ocultas (al enviar)

```
clasificar_velocidad(60) => "normal"  # justo 60
clasificar_velocidad(100) => "rápido"  # justo 100
clasificar_velocidad(101) => "exceso"
clasificar_velocidad(0) => "normal"
```